What Does the Target Leave Behind? Sparse-Autoencoder Concept Pruning for Text–Tabular Learning

Or Davidovich
Technion – Israel Institute of Technology

October 21, 2026

Abstract

In text–tabular learning, each free-text column is embedded and compressed before a tree model sees it. Joint TAR fine-tunes the encoder on the target so that this compression keeps target-relevant variation. We ask whether one can go further by *removing* concepts: we decompose each column's embeddings with a sparse autoencoder (SAE), test every concept against a shuffled copy of itself inside one model with the table and all other concepts, subtract the concepts that fail, and compress what remains. On the 20 text datasets of MulTaBench, pruning removes about 70% of the concepts (38% of the embedding variance) while keeping 80% of TAR's gain over frozen embeddings, but it does not improve on Joint TAR: it is slightly but consistently worse (-0.0035 AUROC/R^2, 4/20 datasets, $p = 0.006$). Choosing the amount by nested cross-fitting, so that every decision is made on rows the encoder never saw, lets the method opt out where removal hurts but yields no gain. [Against random removal of the same variance: . . .] We then test, in one paired run with three learners and multiplicity correction, whether the target helps more when it reweights or adds information instead of deleting it (PLS instead of PCA, no compression, stacking a text model's out-of-fold predictions, late fusion). [Result.]

1 Introduction

Many real tables mix numbers and categories with free text: job ads, product reviews, wine descriptions. A standard recipe embeds each text cell with a pretrained encoder, reduces it to a few dimensions, and gives the result to a gradient-boosted tree model alongside the other columns [7, 11]. MulTaBench [13] evaluates two variants of this recipe. *Joint Frozen* embeds with a frozen E5 encoder [14] and keeps 30 principal components per column; *Joint TAR* first fine-tunes the encoder on the target, so that the principal components follow the variation that matters for the label.

Both variants keep *everything* the text expresses and let compression decide what survives. A product description, however, talks about many things—materials, shipping, brand tone—of which only a few relate to the price. Principal components follow variance, not relevance, so a frequent but irrelevant theme can occupy components that a weaker relevant theme needs. This suggests an explicit alternative: break the embedding into interpretable concepts, remove those unrelated to the target, and only then compress.

Sparse autoencoders (SAEs) provide such a decomposition: they write an embedding as a sparse, non-negative combination of learned directions that tend to be individually interpretable [3, 5, 6]. We use them as a filter rather than as features. Our contributions are:

- A concept-pruning layer for text–tabular pipelines: per-column SAE, joint sibling-aware relevance testing against shuffled "shadow" concepts, and subtractive map-back into the embedding space (§3).

- A leakage-controlled evaluation on all 20 MulTaBench text datasets, with a random-removal control and a nested cross-fitted variant that makes every pruning decision on rows the encoder never saw (§4).
- A negative but informative result: removing concepts does not beat target-aware fine-tuning, even with clean decisions (§5).
- A paired comparison of other ways to use the target on top of TAR—supervised projection, no compression, stacking, late fusion—with LightGBM, tuned LightGBM and CatBoost, and Holm-corrected tests.

2 Related work

Text in tabular learning. Benchmarks of tables with text fields [11, 13] show that tree ensembles over compressed text embeddings are strong baselines. Target-aware text representations appear both in fine-tuning the encoder before compression (Joint TAR, 13) and in tabular foundation models that attend from target tokens to verbalised cells, such as TabSTAR [1].

Supervised dimensionality reduction. Supervised PCA keeps only features correlated with the outcome before extracting components [2]; partial least squares finds directions of maximal covariance with the target [15]. Our method applies the screening idea in an SAE concept basis instead of raw coordinates, and removes concepts instead of selecting coordinates.

Feature relevance with correlated features. Univariate screening and marginal importance are unreliable when features are redundant [12]. Boruta [10] compares each feature with permuted "shadow" copies inside one model; we extend it to groups of sibling SAE concepts. Out-of-fold estimation and cross-fitting [4, 16] motivate our nested variant.

Sparse autoencoders. SAEs are a central tool of mechanistic interpretability [3, 5]; we use TopK SAEs [6]. Prior work uses SAE concepts to interpret or steer models; we test whether they can *filter* representations for a downstream predictor.

3 Method

Pipeline. For each text column c, let $x \in \mathbb{R}^{384}$ be the (frozen or TAR-tuned) E5 embedding of the string "`passage: c: value`". Joint Frozen and Joint TAR fit PCA-30 on the training rows' x and pass the 30 components to LightGBM [9] together with the table. We insert one step before PCA, fitted on the training rows only.

Concept decomposition. We train a TopK SAE [6] with $L = 2048$ latents and $k = 32$ active per input on the column's training embeddings: $z = \mathrm{TopK}(W_e(x - b))$, $\hat{x} = b + \sum_i z_i d_i$ with unit-norm decoder directions d_i. The SAE is unsupervised. Concepts active on fewer than 20 training rows are never tested and are always kept.

Joint, sibling-aware relevance. Testing concepts one at a time fails in two ways: near-duplicate "sibling" concepts (common in large SAEs) each look dispensable because the other covers for it, and concepts that matter only together with the table or other concepts look irrelevant. We therefore (i) cluster eligible concepts by decoder direction (average-linkage cosine distance < 0.5) into sibling groups; (ii) fit one LightGBM on *[table, all concepts, a shadow copy of every group]*, where a group's shadow is its activations with rows permuted jointly; and (iii) drop a group whose total split gain does not exceed its own shadow's in a majority of three repeats.

Map-back. Dropped concepts are subtracted, keeping the SAE's reconstruction error: $\tilde{x} = x - \sum_{i \in \mathcal{D}} z_i d_i$. PCA-30 is then fitted on $\tilde{x}$, so the downstream model sees exactly as many text features as in Joint Frozen/TAR.

Choosing how much to remove. The rule above fixes the amount implicitly. Two variants choose it from data. *Held-out*: rank groups by harm (variance taken divided by importance measured as the rise in loss when the group is permuted on TAR's early-stopping rows) and pick the removal fraction $f \in \{0, .1, .25, .5, .75\}$ by loss on those rows. *Cross-fitted*: split the training rows into three parts; for each part, fine-tune TAR on the other two, rank concepts there (variance taken divided by shadow-corrected gain, from one shadow-test LightGBM), and evaluate every f on the unseen part with a LightGBM on the same inputs as the final model (table and all text columns). The f with the lowest total held-out loss is applied to the fold's ordinary TAR. In both variants $f = 0$ (plain TAR) is a candidate, and a larger removal must lower the loss by more than 0.1%.

Alternatives to removal. On the same TAR embeddings we also test methods that use the target without deleting anything: *PLS-30* in place of PCA-30 [15]; *no compression* (all 384 dimensions); *stacking*, which adds the out-of-fold predictions of a text-only MLP on frozen embeddings as extra columns [16]; and *late fusion*, a logistic/linear meta-learner over the out-of-fold outputs of a table-only LightGBM and the same text model.

4 Experimental setup

Data and protocol. All 20 text datasets of MulTaBench (3 binary, 7 multiclass, 10 regression), with the benchmark's splits (two repeats of 3-fold cross-validation; training sets capped at 10k rows) and metrics: AUROC (one-vs-rest macro for multiclass) and R^2. Joint TAR follows the benchmark recipe: LoRA [8] of rank 16 on the last three E5 layers, a linear head, regression targets binned into 20 quantiles, early stopping on 10% of the training rows (at most 1,000). Within a fold, all TAR-based arms use the *same* fine-tuned encoder; in the final run its LoRA adapters are saved and restored for every arm and learner, so differences isolate what follows the encoder.

Runs and learners. The *main run* evaluates pruning and frozen-embedding variants with LightGBM at default settings, as in the benchmark. The *final run* evaluates all TAR-based arms together with three learners: default LightGBM; LightGBM tuned on its validation split (learning rate, depth, column subsampling; 12 configurations with early stopping); and CatBoost with the benchmark's defaults. The cross-fitted variant was run on five datasets.

Controls. Random pruning removes random eligible concepts from the same SAE, either the same *number* as concept pruning or, because the selected concepts are mostly weak ones, the same *variance*.

Leakage control. Every component that uses the target is fitted on the outer fold's training rows only; test rows are never used. Out-of-fold folds keep rows with identical text together.

Statistics. Methods that use the target are compared with our own Joint TAR on the same splits and learner, using a two-sided Wilcoxon signed-rank test over per-dataset means (folds of a dataset are not independent and only reported as win/loss counts). Since several arms are compared with TAR, we report Holm-adjusted p-values.

5 Results

Removal does not beat TAR. In the main run, concept pruning on top of TAR is slightly but consistently worse than TAR (-0.0035, median -0.0023; better on 4/20 datasets, $p = 0.006$), across

Table 1: Final run, LightGBM: each arm vs. Joint TAR with the same fine-tuned encoder (per-dataset means over 6 folds). p: Wilcoxon over datasets; p_{Holm}: adjusted for the number of arms.

[generated by make_tables.py from the final run]

Table 2: What is removed from TAR embeddings (final run), averaged over datasets, folds and text columns.

[generated by make_tables.py from the final run]

task types (binary -0.0018, multiclass -0.0012, regression -0.0057); the final run gives [Δ, p]. Choosing the amount on TAR's early-stopping rows was worse still on the six datasets where it ran (17 of 23 folds lost): on about a thousand rows, the candidate amounts differ by less than the selection noise. Per-dataset scores are in Appendix A.

How much is removed, and is it the right part? About 70% of the eligible concepts, carrying 38% of the variance of TAR embeddings (the SAE leaves 2% unexplained), at a cost of 0.0035; concept pruning keeps 80% of TAR's gain over Joint Frozen. Removing the same *number* of random concepts is much worse ($+0.0174$ for concept pruning, 18/20), but this control removes 57% of the variance, so it overstates the selection's value. Against random removal of the same variance (Table 2), [result]. [n%] of the removed groups were never split on, either by the concept or by its shadow, so "removed" partly means "unused" rather than "shown irrelevant". Sibling grouping rarely merged concepts (1338 groups for 1348 eligible concepts); redundant concepts inside the joint model share gain rather than being tested as a unit, so the sibling problem is mitigated, not solved.

Clean decisions do not change the picture. The cross-fitted variant (five datasets, Table 6 in Appendix A) is on par with TAR (-0.0006). On Kickstarter and DS salary every removal raises the held-out loss, so it chooses 0% and reproduces TAR exactly; on Baby product prices it chooses moderate amounts and gains slightly. On Fake job postings and Jigsaw it chooses 75% in every fold, which lowers the inner models' held-out log loss by 3–10%, yet the final model is slightly worse than TAR: removal helped encoders fine-tuned on two thirds of the rows, not the one fine-tuned on all of them.

Reweighting and adding instead of removing. [Table 1: TAR+PLS, all dimensions, TAR+stacking, late fusion vs. TAR.] In the main run, stacking on *frozen* embeddings improved on Joint Frozen on 19/20 datasets ($+0.0086$, $p < 0.001$) and was statistically indistinguishable from TAR ($p = 0.55$), whereas PLS on frozen embeddings was not significantly better than PCA ($+0.0044$, 13/20, $p = 0.37$).

Other learners. [Table 3: does the ranking of arms hold with tuned LightGBM and CatBoost?]

6 Limitations and conclusion

We used one encoder (E5-small) and one SAE size. A pilot with TabSTAR [1] as the backbone was infeasible on our hardware (two epochs within its 10-hour budget on one dataset) and is not reported. Cross-fitting costs three extra fine-tunes per fold and was run on five datasets only. In three multiclass runs of the main run (stacking on Spotify, two folds; held-out pruning on Wine, one fold) the final model collapsed to chance level (test log loss > 29); we have not identified the cause and report them unchanged.

[Conclusion, once the final run is in.] Concept pruning separates concepts the model uses from those it does not, but after target-aware fine-tuning removing the rest does not improve prediction.

Table 3: The same comparisons with other learners (final run): mean difference vs. that learner's Joint TAR, datasets won, Wilcoxon p.

[generated by make_tables.py from the final run]

Code. [Repository URL]. All results can be regenerated with `sae_pilot.py`; tables with `paper/make_tables.py`.

References

[1] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A foundation tabular model with semantically target-aware representations. *arXiv preprint arXiv:2505.18125*, 2025.

[2] Eric Bair, Trevor Hastie, Debashis Paul, and Robert Tibshirani. Prediction by supervised principal components. *Journal of the American Statistical Association*, 101(473):119–137, 2006.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[4] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018.

[5] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[6] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[7] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS Datasets and Benchmarks Track*, 2022.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[9] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.

[10] Miron B. Kursa and Witold R. Rudnicki. Feature selection with the Boruta package. *Journal of Statistical Software*, 36(11):1–13, 2010.

[11] Xingjian Shi, Jonas Mueller, Nick Erickson, Mu Li, and Alexander J. Smola. Benchmarking multimodal AutoML for tabular data with text fields. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[12] Carolin Strobl, Anne-Laure Boulesteix, Thomas Kneib, Thomas Augustin, and Achim Zeileis. Conditional variable importance for random forests. *BMC Bioinformatics*, 9:307, 2008.

[13] TODO: authors – copy from arXiv 2605.10616. MulTaBench: Benchmarking multimodal tabular learning with text and image. *arXiv preprint arXiv:2605.10616*, 2026.

[14] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[15] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58(2):109–130, 2001.

[16] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

A Per-dataset results

Table 4: Mean AUROC (classification) or R^2 (regression) over 6 folds, LightGBM. Best per row in bold. †: fewer than 6 folds.

Dataset	Frozen	TAR	TAR+prune	TAR+rand	PLS	Stack
Fake job postings (bin)	0.914	0.911	0.915	0.903	0.907	**0.920**
Jigsaw toxicity (bin)	0.819	**0.933**	0.929	0.880	0.864	0.864
Kickstarter (bin)	0.718	**0.734**	0.728	0.708	0.724	0.727
DS salary (mul)	0.798	**0.807**	0.803	0.797	0.793	0.806
Michelin (mul)	0.893	**0.904**	0.903	0.901	0.877	0.902
Product sentiment (mul)	0.874	**0.890**	0.887	0.889	0.846	0.879
Spotify genres (mul)	0.900	0.904	**0.905**	0.903	0.900	0.794
US accidents (mul)	0.970	**0.978**	**0.978**	0.975	0.970	0.973
Wine reviews (mul)	0.966	**0.975**	**0.975**	0.974	0.971	0.974
Women's clothing (mul)	0.898	**0.906**	0.904	0.903	0.896	0.902
Baby product prices (reg)	0.777	0.795	0.789	0.757	**0.808**	0.808
Book prices (reg)	0.496	0.514	0.499	0.469	0.501	**0.521**
Book readability (reg)	0.805	**0.815**	0.813	0.803	0.809	0.807
Mercari (reg)	0.414	0.439	0.433	0.409	0.437	**0.445**
Montgomery salaries (reg)	0.973	0.975	0.975	**0.976**	0.975	0.974
Rotten Tomatoes (reg)	0.494	0.506	0.500	0.474	0.480	**0.509**
SCImago impact (reg)	0.843	**0.849**	0.847	0.843	0.839	0.845
Vancouver salaries (reg)	0.734	0.765	0.768	0.756	**0.769**	0.766
Video game sales (reg)	0.368	0.379	0.355	0.255	0.369	**0.384**
Zomato (reg)	0.783	0.802	0.802	0.788	0.791	**0.809**

Table 5: Final run, LightGBM: mean score per dataset for each TAR-based arm.

[generated by make_tables.py from the final run]

Table 6: Nested cross-fitted removal amount (five datasets). Every decision is made on rows the judging encoder never saw.

Dataset	TAR	TAR + cross-fitted pruning	chosen removal (per fold)
Fake job postings	0.912	0.911	75, 75, 75, 75, 75, 75 %
Jigsaw toxicity	0.931	0.927	75, 75, 75, 75, 75, 75 %
Kickstarter	0.734	0.734	0, 0, 0, 0, 0, 0 %
DS salary	0.807	0.807	0, 0, 0, 0, 0, 0 %
Baby product prices	0.791	0.794	50, 50, 10, 25, 0, 25 %